

# CrashGuard: Autonomous Vehicle Crash Detection with AI-Mediated Emergency Response on the MYOSA Platform

Aidan Garcia, Jacob Randall, Cabe Robertson, Vu Ha Nguyen  
Klipsch School of Electrical and Computer Engineering  
New Mexico State University, Las Cruces, NM, USA  
Faculty Mentor: Dr. Lavrova

**Abstract**—Delayed emergency response is a major contributor to preventable deaths in vehicle collisions, particularly when the driver is incapacitated and unable to call for help. This paper presents CrashGuard, a low-cost autonomous crash detection and response system built on the MYOSA (Make Your Own Sensor Applications) IoT platform. An ESP32-based MYOSA motherboard continuously samples an MPU6050 accelerometer at 100 Hz, normalizes acceleration magnitude against a boot-time baseline, and flags high-magnitude events consistent with a collision. A ten-second occupant-cancelable countdown—rendered on a live smartphone dashboard with an onboard audible alarm, since the vehicle is in motion—suppresses false positives. If the countdown expires, the board transmits the event over Bluetooth Low Energy to a host bridge, which triggers an automated outbound telephone call. A large language model agent, grounded exclusively in a preloaded occupant profile and live crash data, speaks the victim’s critical information—identity, GPS location, blood type, medical conditions, and emergency contacts—and answers the dispatcher’s follow-up questions conversationally. Map data retrieved at the moment of impact extends the agent’s knowledge beyond the stored profile to cross streets, municipality, and the nearest hospitals, and the system offers to deliver a complete written incident report by SMS or email. The complete pipeline is demonstrated on a remote-controlled car and validated through an offline test harness, a console rehearsal tool, and a data-driven threshold calibration procedure.

**Index Terms**—crash detection, emergency response, IoT, MYOSA, ESP32, accelerometer, Bluetooth Low Energy, large language models, conversational AI

## I. INTRODUCTION

Every year, thousands of car crash victims suffer preventable deaths due to delayed emergency response, particularly in cases where the driver is incapacitated and unable to call for help. CrashGuard directly addresses this life-threatening gap by autonomously identifying collision events through sudden changes in acceleration and immediately contacting emergency services without requiring any action from the occupant. By leveraging the MYOSA board’s accelerometer and Bluetooth capabilities, the system provides a low-cost solution that can, in principle, be integrated into any vehicle. The project demonstrates how accessible IoT platforms can be applied to real-world emergencies, potentially saving lives through faster response times.

Commercial telematics services such as OnStar pair automatic crash notification with a human advisor. CrashGuard

explores the same concept with two substitutions appropriate to an educational IoT platform: a modular sensor kit in place of OEM vehicle integration, and a language-model voice agent in place of the human advisor. The agent both reports the occupant’s preloaded vital information and answers the dispatcher’s questions in natural conversation, while remaining strictly grounded in verified data.

## II. SYSTEM ARCHITECTURE

Fig. 1 shows the end-to-end pipeline. The MYOSA motherboard acts as the “brain” of the system: it samples the accelerometer, manages the cancel countdown and buzzer, and streams both crash alerts and live status off-board. Because the demonstration vehicle is moving, no display is mounted on it; all results are rendered on a smartphone. A host-side Python bridge (standing in for the occupant’s smartphone) subscribes to the board’s BLE crash-event and status characteristics, forwards both to a Flask call server, and relays dashboard commands back to a write characteristic on the board. The server additionally serves a mobile web dashboard showing live impact force, session peak, system state, and—during the cancel window—a full-screen countdown with a prominent cancel control; a re-arm control appears after an alert. Cancellation may therefore come from the phone or from an onboard backup pushbutton. The server initiates an outbound telephone call through the Twilio Programmable Voice API, speaks an emergency briefing, and then loops dispatcher speech through a Claude language-model agent to generate grounded spoken answers. A compile-time option lets the board bypass BLE and deliver the alert directly over Wi-Fi HTTP, providing a fallback for congested conference radio environments.

## III. HARDWARE DESIGN

CrashGuard uses the MYOSA Mini kit [1]: an ESP32-based motherboard with Wi-Fi and Bluetooth, an MPU6050 accelerometer/gyroscope module (I<sup>2</sup>C address 0x69) [3], and connected over the kit’s plug-and-play 4-wire JST I<sup>2</sup>C chain at 100 kHz. A passive piezo buzzer and a momentary pushbutton are the only additions; the ESP32 boot button doubles as the backup disarm button for benchtop work. The kit’s SSD1306 OLED module [4] is supported behind a compile-time flag for bench debugging but is not used in the demonstration, where

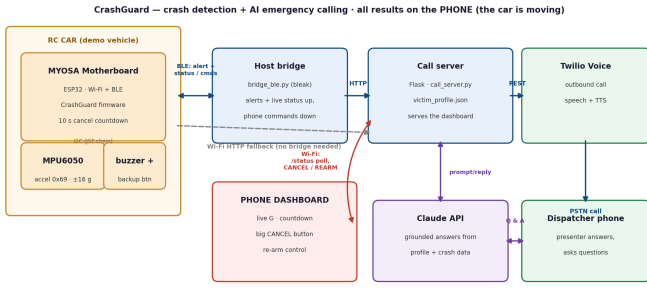


Fig. 1. CrashGuard system architecture. Solid arrows show the primary BLE alert path; the dashed arrow shows the optional Wi-Fi HTTP fallback.

the phone dashboard is the user interface. The kit’s APDS9960 light/proximity/gesture module optionally provides a short-range forward-obstacle warning as a demonstration flourish.

For the demonstration vehicle, the MYOSA board is contained within a protective enclosure mounted to a remote-controlled car, housing the board, its modular sensors, and the disarm button, with power supplied by a compact USB battery pack. The modular nature of the MYOSA kit allows the platform to be physically integrated into an existing mechanical system with minimal expense or effort.

#### IV. CRASH DETECTION ALGORITHM

The firmware configures the MPU6050 for a  $\pm 16$  g full-scale range so that collision transients are not clipped, and samples all three axes at  $f_s = 100$  Hz. The acceleration magnitude is

$$|a(t)| = \sqrt{a_x^2(t) + a_y^2(t) + a_z^2(t)}. \quad (1)$$

At boot, the system averages  $N = 100$  samples while the vehicle is stationary to obtain the baseline  $|a|_{\text{base}}$ , i.e., the sensor’s own reading of 1 g under the current mounting and scale factors. All subsequent readings are normalized:

$$G(t) = \frac{|a(t)|}{|a|_{\text{base}}}. \quad (2)$$

This self-calibration makes the trigger threshold independent of library scale factors, sensor tolerances, and mounting orientation. An impact is declared when

$$G(t) \geq G_{\text{th}} \quad \text{for } n \geq n_{\text{min}} \text{ consecutive samples,} \quad (3)$$

with  $G_{\text{th}}$  and  $n_{\text{min}}$  tuned empirically for the specific platform (Section XI). Readings are passed through a median-of-three filter before comparison, which discards isolated outlier samples arising from bus glitches or transient sensor saturation, so that both the reported magnitude and the trigger reflect sustained motion.

##### A. Sensor Acquisition

Obtaining trustworthy acceleration from the platform required departing from the vendor library’s per-axis accessors. Inspection of the library source revealed three properties unsuitable for high-rate crash sensing. First, each per-axis

accessor re-reads the range configuration register, tripling I<sup>2</sup>C traffic at a 100 Hz sampling rate and yielding intermittently corrupted samples. Second, the three axes are read in separate transactions and therefore correspond to different instants, so their vector magnitude is not physically meaningful during rapid motion—empirically this produced orientation-dependent readings of 14–17 g while the board was stationary. Third, the library’s scale factor is twice the correct value for every full-scale range, applying  $2^{f_{sr}+1}$  where the MPU6050 datasheet requires  $2^{f_{sr}}$ .

The firmware therefore uses the library’s burst accessor, which retrieves all six accelerometer bytes in a single transaction so that the three axes are sampled simultaneously, and applies the datasheet sensitivity (2048 LSB/g at  $\pm 16$  g) directly to the raw counts. Gyroscope data is not read at any point; angular rate is unnecessary for impact detection and its acquisition would only add bus load.

##### B. Gravity Removal and Directional Impact

Rather than thresholding the raw magnitude, the firmware records the gravity *vector*  $\mathbf{g}_0$  during the still-boot calibration and subtracts it from every subsequent sample, yielding the dynamic acceleration

$$\mathbf{d}(t) = \frac{\mathbf{a}(t) - \mathbf{g}_0}{|\mathbf{g}_0|}, \quad (4)$$

which is zero at rest *in any orientation*, because the gravity vector rather than merely its magnitude is removed. The trigger threshold is therefore expressed purely as excess impact acceleration, with no gravity offset buried in the quantity. Since rotating the sensor by hand can produce at most 2 g of apparent dynamic acceleration, a threshold above this value renders ordinary handling incapable of triggering an alert. The dominant axis of  $\mathbf{d}$  and the sign of its component are recorded at trigger time, distinguishing deceleration (the vehicle striking an object ahead) from acceleration (being struck), and this classification is passed to the response agent.

##### C. Sensor Saturation as a Detection Signal

The MPU6050 saturates at  $\pm 16$  g per axis, bounding the reportable magnitude at  $16\sqrt{3} \approx 27.7$  g. Thresholds at or above this value are unreachable—an important practical constraint, since impacts on a rigid demonstration platform readily exceed the sensor’s range. The firmware therefore treats sustained per-axis railing as an independent crash indicator: when an axis reaches 95% of full scale for  $n_{\text{min}}$  consecutive samples, an impact is declared regardless of the computed magnitude, and the alert is annotated to indicate that true severity exceeds the recorded peak. Collisions harder than the instrument can measure are thus detected rather than missed.

A four-state machine governs behavior: MONITORING (armed; live and peak  $G$  stream to the phone dashboard at 5 Hz), COUNTDOWN (impact detected; a ten-second cancel timer with escalating buzzer cadence, mirrored as a full-screen countdown and cancel control on the phone), ALERTSENT (countdown expired; the crash event, including recorded peak

$G$ , is pushed to the host, and the dashboard reports that the emergency call is in progress), and a brief DISARMED pause when the occupant cancels. Re-arming is available from the dashboard or by a long press of the onboard button. Remote cancellation latency is bounded by the status streaming period (200 ms) plus link latency, comfortably inside the ten-second window.

A deliberate engineering decision: the system detects crashes from acceleration magnitude rather than attempting to track vehicle velocity. Integrating consumer-grade accelerometer data yields rapidly drifting velocity estimates, whereas a sudden high-magnitude acceleration change is a robust, low-cost collision signature; the equivalent quantity in production automatic crash notification systems is the impact severity ( $\Delta v$ ) inferred at the moment of collision.

## V. AI-MEDIATED EMERGENCY RESPONSE

When the alert reaches the call server, Twilio places an outbound call to a preconfigured emergency number [6]. The call opens with a spoken briefing assembled from the occupant profile: full legal name, GPS coordinates and nearest street address, recorded peak impact, blood type, medical conditions, and the primary emergency contact.

The dispatcher’s speech is then transcribed by Twilio’s speech recognition and forwarded, with the full call history, to a Claude language-model agent [7]. The agent’s system prompt embeds the occupant profile and live crash data and imposes strict behavioral rules: answer in one to three short spoken sentences; read telephone numbers digit by digit and coordinates slowly; and respond “I do not have that information” rather than speculate—the agent cannot sense the cabin and must never invent injury details. Preconfigured personal information is stored in a simple JSON profile that the user completes during setup.

For safe demonstration, three safeguards apply: the destination number is always a team member’s phone, never a real emergency service; a demo-mode flag instructs the agent to state clearly that no real emergency exists if asked; and a server-side cooldown suppresses duplicate triggers. A mock agent mode reproduces the full call flow offline with keyword-matched answers, allowing the pipeline to be exercised with no network accounts.

## VI. UTILIZATION OF THE MYOSA PLATFORM

The MYOSA board is a powerful IoT environment that enables rapid prototyping of real-world embedded systems. This project leverages the platform by treating the MYOSA board as the central intelligence of an autonomous crash detection system, exploiting both its onboard sensing capabilities and its wireless communication features simultaneously. The application is built upon the accelerometer by continuously sampling acceleration data, establishing a baseline, and flagging any sudden, high-magnitude change consistent with a collision, while the built-in Bluetooth radio communicates in real time with the host that places the call. Together, these elements demonstrate that the MYOSA platform is capable

of powering end-to-end IoT applications—from raw sensor input, through onboard processing, to real-world automated response—making it a compelling foundation for life-saving technology.

## VII. MAP-DERIVED SCENE CONTEXT

A dispatcher’s questions extend beyond any preloaded profile: the nearest cross street, the municipality, the closest emergency department. These depend on where the collision actually occurred and therefore cannot be stored in advance.

At trigger time, before the call is placed, CrashGuard issues a single batch of queries against OpenStreetMap services—Nominatim for reverse geocoding and Overpass for nearby features—using the live coordinates supplied by the occupant’s phone. The result is a compact scene bundle: street address, city, county and state, the nearest named roads ordered by distance, and hospitals within a 15 km radius annotated with great-circle distance and compass bearing computed by the haversine formula.

This bundle is fetched *once* and injected into the language model’s context. The design has three consequences. Latency is bounded, because no network lookup occurs during the conversation. Cost is bounded, because the agent never requires an additional round trip to retrieve a fact. And correctness is preserved, because the agent reads authoritative map data rather than inferring street names, consistent with the grounding rules of Section V.

## VIII. AUTOMATED INCIDENT DOCUMENTATION

After several exchanges the agent offers to document the incident. On acceptance it asks whether delivery should be by text message or electronic mail and transmits a formatted report containing the detection timestamp, peak impact magnitude, coordinates and address, cross streets, nearest hospitals, occupant identity and medical data, emergency contacts, and vehicle description. Delivery uses the existing Twilio number for SMS or SMTP for email.

The offer, the affirmative/negative determination, and the channel selection are resolved by local intent parsing rather than by the language model, so the feature contributes no additional inference cost and responds without model latency.

## IX. COST AND LATENCY OPTIMIZATION

Three measures reduce the per-call inference cost, which is dominated by the system prompt being re-billed on every conversational turn. First, the system prompt is marked for ephemeral prompt caching, so the invariant profile and scene data are billed at the reduced cached rate after the first turn. Second, conversation history is truncated to the most recent turns, bounding context growth over a long call. Third, the prompt itself was rewritten in compact form and the reply budget capped at 100 tokens, consistent with the one-to-two-sentence spoken replies a dispatcher needs. Latency is additionally reduced by dispatching model inference to a worker thread and returning to the telephony provider immediately, so that provider-side webhook timeouts cannot terminate the call during a slow inference.

TABLE I  
CALIBRATION RESULTS (TO BE COMPLETED FROM RC-CAR RUNS)

| Run type                         | Runs      | Max $G$ | p99.9 $G$ |
|----------------------------------|-----------|---------|-----------|
| Normal driving                   | TBD       | TBD     | TBD       |
| Simulated collision              | TBD       | TBD     | TBD       |
| Selected $G_{th}$                | TBD       |         |           |
| False alarms / missed detections | TBD / TBD |         |           |

## X. DEMONSTRATION PLAN

The conference demonstration uses an RC car carrying the enclosed MYOSA board. The presenter first drives the car normally, showing live impact data on the OLED and the board’s Bluetooth link to the host. The presenter holds a phone showing the live dashboard—mirrored on the projector—with real-time impact force. The car is then driven into a soft target to produce a simulated collision: the dashboard flashes the cancel countdown while the buzzer on the car sounds. In one pass the presenter taps the dashboard’s cancel control, showing false-alarm handling; in the second, the countdown is allowed to expire. The board alerts the host, and the system places a live call to the presenter’s phone over a speaker: the AI reports the preconfigured occupant information—GPS coordinates, full legal name, blood type, emergency contacts, and additional medical details—and then answers dispatcher-style questions asked by the presenter or by audience volunteers. If the kit’s proximity module is enabled, the car additionally emits a forward-obstacle warning when approaching objects.

## XI. TESTING AND CALIBRATION

Three test layers precede any hardware run. First, an offline harness exercises the complete server in-process—trigger, briefing, three dispatcher question/answer rounds, closing, silence handling, and the full dashboard pipeline (status ingestion, command queuing, and both command delivery paths)—using the mock agent; all twenty-five checks pass with no network access. Second, a console rehearsal tool replays the identical prompt and data over the keyboard, allowing the team to tune the occupant profile and rehearse dispatcher interactions (with the mock agent or the live model). Third, a crash simulator posts synthetic events to the server to validate live telephony before the board is involved.

Threshold selection is data-driven. The firmware streams  $T, \text{millis}, G, \text{peak}, G, \text{state}$  telemetry over USB serial at 20 Hz; a calibration script records normal-driving runs (aggressive starts, stops, turns, and bumps) and deliberate crash runs, then reports the maximum and 99.9th-percentile  $G$  for each. The threshold is placed at  $G_{th} \geq 1.5 \times G_{\max}^{\text{normal}}$  and  $G_{th} \leq 0.7 \times G_{\text{peak}}^{\text{crash}}$ . Table I is completed from the team’s calibration data.

## XII. CONCLUSION

CrashGuard demonstrates that a complete automatic-crash-notification pipeline—sensing, occupant interaction, wireless alerting, autonomous telephony, and conversational information relay—can be assembled from an educational IoT kit,

commodity cloud APIs, and open-source software. The architecture cleanly separates the safety-critical embedded path (detect, confirm, alert) from the information-rich cloud path (call, brief, answer), so each can be tested independently. Natural extensions include a smartphone app replacing the laptop bridge to contribute live GPS, gyroscope-based rollover detection, multilingual dispatcher support, and a cellular (LTE-M) alert path for fully standalone operation.

## ACKNOWLEDGMENT

The authors thank their faculty mentor Dr. Lavrova and the Klipsch School of Electrical and Computer Engineering at New Mexico State University, the NMSU IEEE Student Branch, and the MYOSA initiative of MakeSense EduTech and Pegasus Automation, supported by the IEEE Sensors Council, for the MYOSA Mini kit and the opportunity to participate in MYOSA Event 6.0.

## REFERENCES

- [1] MYOSA, “MYOSA Mini Kit,” [Online]. Available: <https://myosa-sensors.org/kits/myosa-mini/>
- [2] MYOSA, “Arduino libraries and example codes,” [Online]. Available: <https://github.com/myosa-sensors/arduino-libraries>
- [3] InvenSense Inc., “MPU-6000 and MPU-6050 Product Specification,” Rev. 3.4, 2013.
- [4] Solomon Systech, “SSD1306 128×64 Dot Matrix OLED Segment/Common Driver with Controller,” datasheet, 2008.
- [5] Espressif Systems, “ESP32 Series Datasheet,” [Online]. Available: <https://www.espressif.com>
- [6] Twilio Inc., “Programmable Voice API documentation,” [Online]. Available: <https://www.twilio.com/docs/voice>
- [7] Anthropic, “Claude API documentation,” [Online]. Available: <https://docs.claude.com>
- [8] OpenStreetMap Foundation, “Nominatim geocoding service” and “Overpass API,” [Online]. Available: <https://www.openstreetmap.org>